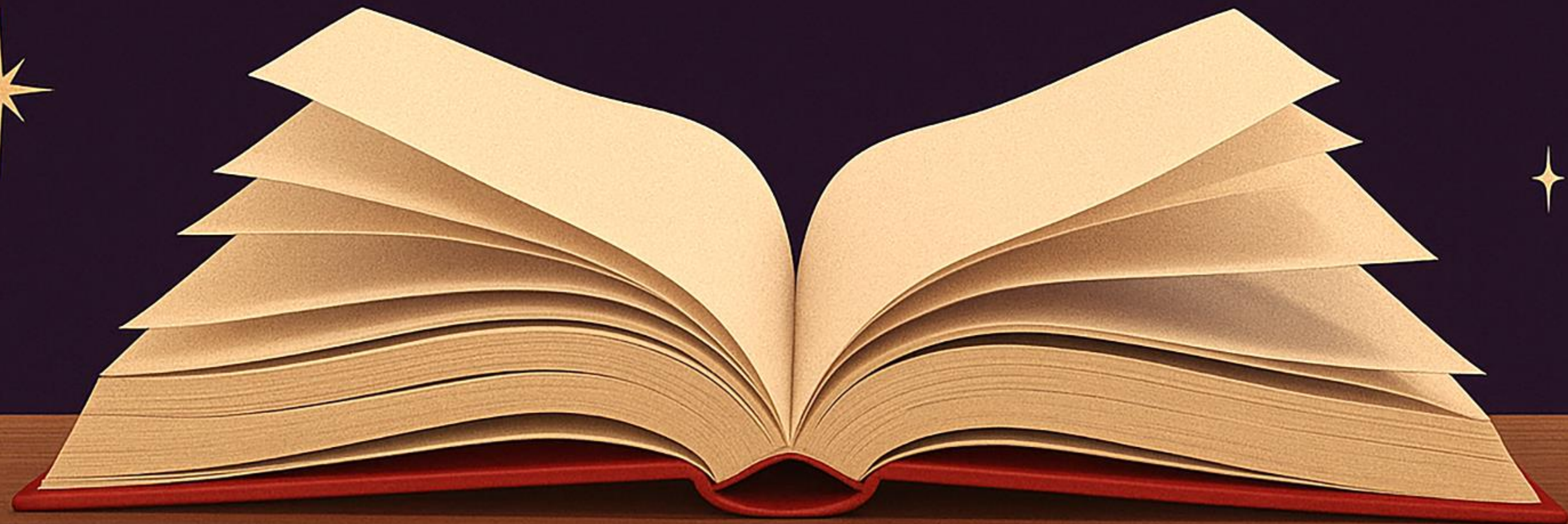


Book Talk:
영화를 읽다,
Movie Log



목차

- 기획 의도 및 컨셉
- 차별적 핵심기능
- 업무 수립 계획
- ERD
- SW 시스템 아키텍처
- 기술스택
- 기능 소개
- 시연 영상
- 개인별 자체평가

영화감상은 다양한 기록을 눈으로 관찰하는 것과 같다.

기획 의도 및 컨셉



책은 영화를 그리게 합니다.

'책'은 역사적 기록 및 사건, 사회시사 및 풍자, 전문적 지식과 근거, 판타지 소설 등을 '인쇄매체'로서 접합니다. '영화'는 같은 내용을 '영상매체'로서 접하는 것일뿐 사람들이 취하는 정보는 동일합니다.

책은 청각적, 시공간적 정보를 담지 못하는 한계를 지니고 있기 때문에 독자의 상상력을 통해 오감정보를 느끼도록 진입한다는 점에서 '**책 속의 영화**'라는 컨셉을 기획하게 되었습니다.

타사 대비 차별적 핵심기능



직관적이고 빠른 예매 시스템

단순한 구조와 디자인으로 남녀노소 빠르고 간편하게 예매할 수 있습니다



더욱 편리한 관리자 시스템

관리자 또한 사용자이므로 영화를 등록하거나 삭제할 때 더욱 직관적이고 효율적이게 구성하였습니다



VIP 회원 채팅 시스템(VIP LOUNGE)

영화 예매시 포인트를 지급하여 개별등급을 설정하고 500포인트 이상부터 VIP간의 영화평론을 주고 받을 수 있는 커뮤니케이션 시스템을 도입했습니다.

효율적인 업무 프로세스를 위한

업무 수립 계획

1

견고한 최초 기획, 목적 지향성 강화

어떤 프론트를 제작할지 Figma를 통해 협업하여 구조 및 디자인을 설계하고 계획적으로 진행하여 개발 중 불필요한 시행착오를 감소합니다.

2

개인의 능력을 고려한 업무 분담

팀원들의 유형이 기획형, 설계형, 기술형으로 나뉘는 것을 발견했습니다. 이를 고려해 업무를 분담하여 각자의 역량을 최대한 발휘할 수 있도록 효율성을 극대화합니다.

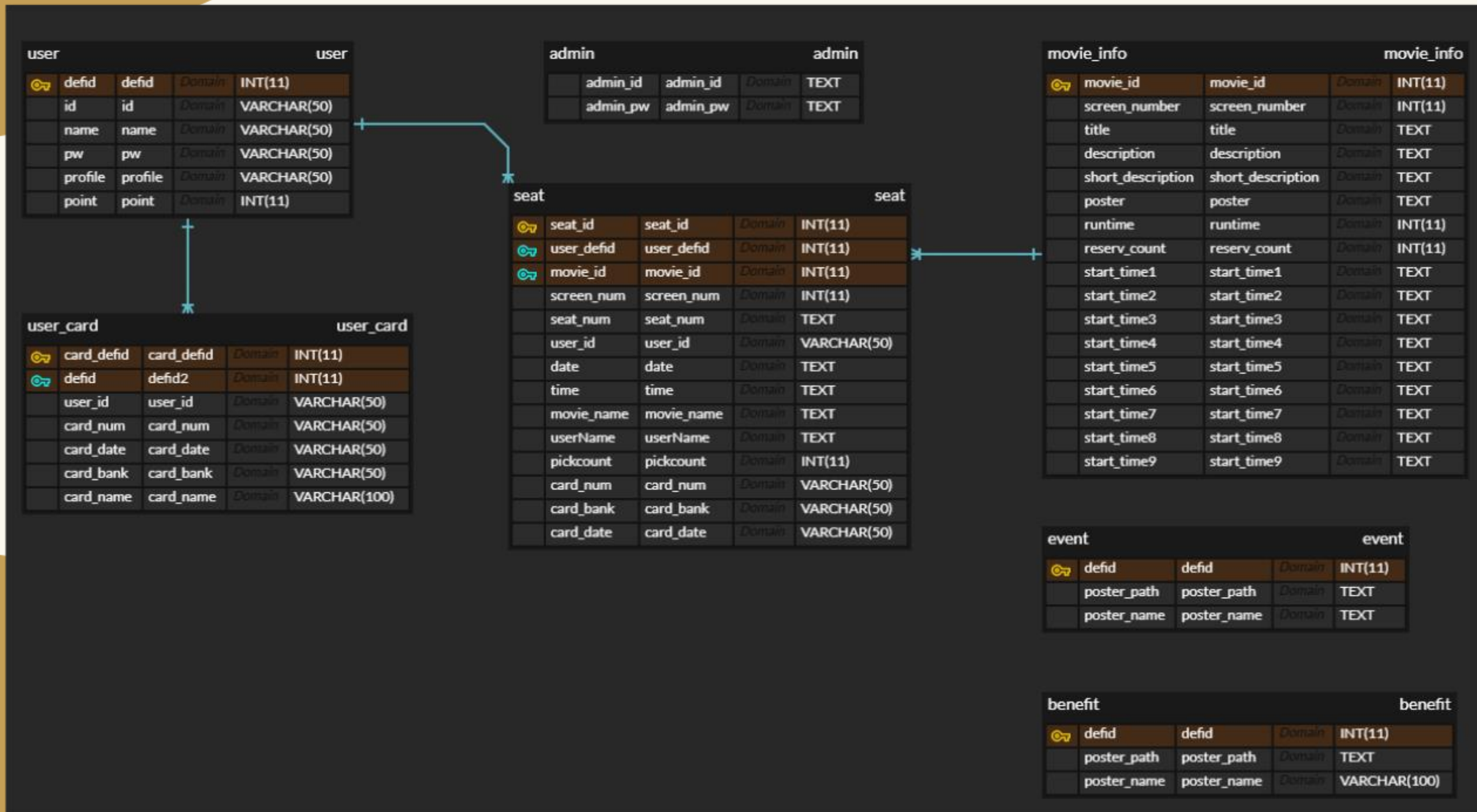
3

철저한 오류 검수

웹사이트 기능을 다회 반복하여 예기치 못한 오류를 최대한 발견하여 수정하고 코드 리팩토링, 리팩터링을 통해 유지보수성을 증진시킵니다.

대제	소제	작업 기간	작업자
기획	브랜드 아이덴티티 설정	25.12.02 - 25.12.02	유조현
	업계 경쟁력 분석	25.12.02 - 25.12.02	김찬
	영화관 규모 설정	25.12.02 - 25.12.02	김찬
구조 설계	브랜드 아이콘 제작	25.12.02 - 25.12.02	유조현
	메인페이지	25.12.02 - 25.12.02	유조현
	로그인 페이지	25.12.02 - 25.12.02	김찬
	회원가입 페이지	25.12.02 - 25.12.02	김찬
	영화 페이지	25.12.02 - 25.12.02	김찬
	예매 페이지	25.12.02 - 25.12.02	유조현
	이벤트 페이지	25.12.03 - 25.12.03	유조현
	혜택 페이지	25.12.03 - 25.12.03	유조현
	마이페이지	25.12.03 - 25.12.03	김찬
	관리자 페이지	25.12.04 - 25.12.04	유조현
	ERD 설계	25.12.03 - 25.12.03	김찬
	메인헤더	25.12.04 - 25.12.04	유조현
디자인	로그인 페이지	25.12.04 - 25.12.04	김찬
	회원가입 페이지	25.12.04 - 25.12.04	김찬
	영화 페이지	25.12.04 - 25.12.04	김찬
	예매 페이지	25.12.04 - 25.12.04	유조현
	이벤트 페이지	25.12.04 - 25.12.04	유조현
	혜택 페이지	25.12.04 - 25.12.04	유조현
	마이페이지	25.12.05 - 25.12.05	김찬
	관리자 페이지	25.12.04 - 25.12.09	곽재범
	메인페이지 영화정보 조회 및 페이지 넘김	25.12.08 - 25.12.08	유조현
	로그인	25.12.05 - 25.12.08	곽재범
기능 구현	로그아웃	25.12.05 - 25.12.08	곽재범
	회원가입	25.12.05 - 25.12.08	곽재범
	메인헤더 로그인 상태 반영	25.12.05 - 25.12.08	곽재범
	영화 페이지 영화정보 조회	25.12.08 - 25.12.08	김찬
	예매 페이지 좌석예매 기능	25.12.08 - 25.12.09	유조현
	혜택 페이지 포스터 조회	25.12.09 - 25.12.09	유조현
	이벤트 페이지 포스터 조회	25.12.09 - 25.12.09	유조현
	SQL 구축	25.12.05 - 25.12.08	김찬
	SQL 기반 연동	25.12.09 - 25.12.09	유조현
	사용자 카드 관리 기능	25.12.09 - 25.12.09	유조현
	결제 페이지	25.12.09 - 25.12.09	김찬
	개인정보 수정	25.12.09 - 25.12.09	유조현
	사용자 예매내역	25.12.09 - 25.12.09	김찬
	프로필사진	25.12.08 - 25.12.09	김찬
	사용자 누적 포인트	25.12.10 - 25.12.10	김찬
	VIP LOUNGE 채팅방 개설	25.12.10 - 25.12.10	김찬
	관리자 페이지 헤더 및 로그인	25.12.05 - 25.12.10	곽재범
	관리자 페이지 관리 기능	25.12.10 - 25.12.10	유조현
	소프트웨어 테스트	25.12.10 - 25.12.11	유조현
검수	의존성 배열 무한호출 해결	25.12.11 - 25.12.11	유조현
	관리자 페이지 영화 등록 무반응	25.12.12 - 25.12.12	유조현
	포스터 업로드 무반응	25.12.11 - 25.12.11	김찬
	좌석 예매수 과다시 레이아웃 이탈	25.12.12 - 25.12.12	유조현
	코드 리팩토링 및 리팩터링	25.12.12 - 25.12.12	유조현

데이터베이스 구조 (ERD)



주요 관계 구조

- 관리자 페이지에서 전체 통제 가능
단. 관리자 계정 관리의 보안상 데이터베이스 담당자와 연락해서 신분확인 절차 후 진행
- 사용자 삭제시 관련 정보 일괄 삭제
사용자 탈퇴 및 관리자 권한으로 삭제시 사용자 관련 카드 정보 및 예매 정보를 일괄 삭제하여 효율적으로 데이터베이스 관리
- 영화정보 삭제시 해당 예매기록 일괄 삭제
영화 상영이 종료된 후 관리자가 해당 영화를 삭제만 하면 예매기록도 일괄 삭제되어 데이터베이스 관리가 용이



사용자 및 관리자 웹 접근

사용자가 웹 브라우저에 접근하여 다양한 기능 및 페이지 이동을 진행합니다.



React SPA Frontend

Single Page Application 구조로서 최초 1회 HTML을 로드한 후 화면 전환을 클라이언트에서 처리합니다.



Node Express API

Frontend에 필요한 데이터를 API 형태로 서버를 구축하여 활용합니다.



Maria DB 및 File Storage

파일 업로드시 별도의 Stotage 폴더에 저장되며 데이터베이스에서 경로를 불러오는 방식으로 설계합니다.



WS Server Chat

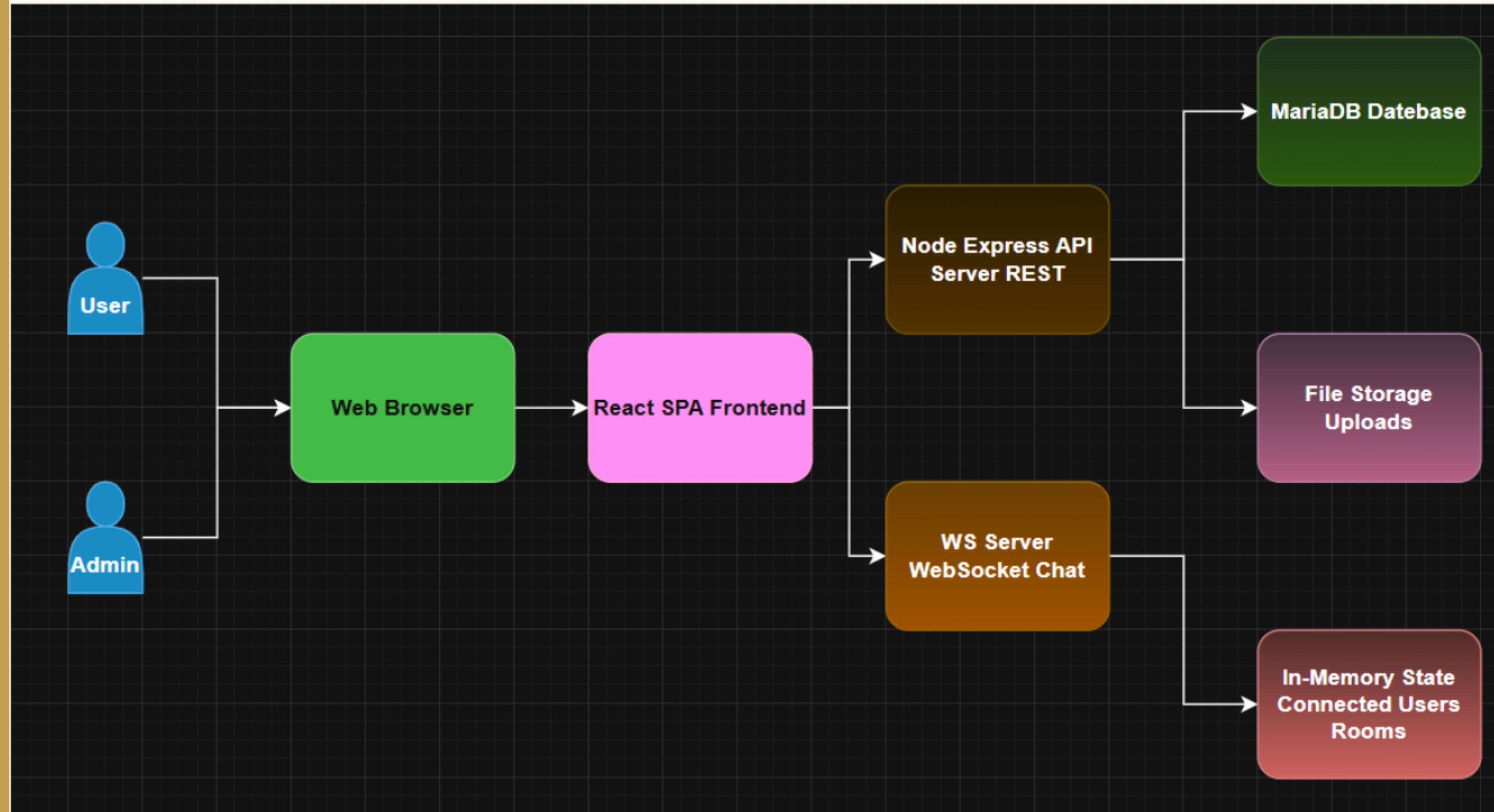
WebSochet 프로토콜을 사용하여 클라이언트와 서버 간 실시간 양방향 통신을 구현하였습니다.



In-Memory State

채팅 기록은 저장하지 않고, 현재 접속 중인 사용자 정보와 채팅방 상태를 서버 메모리에서 관리합니다.

SW 시스템 아키텍처



기술 스택

Front-end



IDE



Collaboration



Back-end



DBMS



Tools

DB Client Tool



Design Tool



접속시 : 팝업 동영상 광고

```
const [showAd, setShowAd] = useState(false)

useEffect(() => { //영화정보 조회
  fetch("http://192.168.0.228:3000/movieinfo")
    .then(response => response.json())
    .then(data => setMovieData(data))
}, [movieData])

useEffect(() => {
  const lastClosed = localStorage.getItem("adClosedAt");

  if (!lastClosed) {
    setShowAd(true); // 광고 본 적 없음 → 보여줌
    return;
  }

  const diff = Date.now() - Number(lastClosed);

  if (diff > 24 * 60 * 60 * 1000) {
    // 24시간 지남 → 다시 광고 보여줌
    setShowAd(true);
  }
}, []);
```

```
function closeAd(){
  localStorage.setItem("adClosedAt", Date.now());
  setShowAd(false)
}
```

```
return (
  <>
    <MainHeader />

    <main className="main-area">
      { /* 전체 화면 광고 모달 */ }
      { showAd && (
        <div className="ad-modal">
          <div className="ad-content">

            <video
              src={ad}
              autoPlay
              muted
              loop
              className="ad-video"
            />

            <button className="ad-close-btn" onClick={closeAd}>
              close ad
            </button>
          </div>
        </div>
      ) }
    </div>
  )
)
```

- 광고 노출 조건
showAd = 광고 노출 조건 판단

closeAd = 광고 닫기 버튼 클릭시
localStorage에 { adCloseAt : 현재
시각 }을 set

diff = 접속시 현재 시간 - lastClosed
즉 localStorage에 보관했던 시간 정
보와 현재 시간의 차를 구함.

(24*60*60*1000) = 24시간
즉. 광고를 담은지 24시간 차 발생시
showAd를 true로 set하고

showAd가 true일 경우 video태그
를 생성하기 때문에 팝업 광고가 24시
간 단위로 노출됨.

- 팝업화 CSS
position: fixed;
=> video 태그 팝업화
backdrop-filter: blur(6px);
=> video 후방 블러처리



```
/* ===== 예고편 광고 팝업(모달) ===== */
.ad-modal{
  position: fixed;
  inset: 0;
  z-index: 9999;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 24px;
  background: rgba(0,0,0,0.65);
  backdrop-filter: blur(6px);
}
```


메인 : 예매누적수 기반

```
//영화정보 호출 (예매누적수 기반 내림차순)
app.get('/movieinfo', async (req, res) => {
  const data = await pool.query(
    'SELECT * FROM movie_info ORDER BY reserv_count DESC')
  res.send(data)
})
```

	12월	12월	AZ title	A번	A번	A번	12월	12월 reserv_count
1	17	1	주토피아	디즈	더 화	/uplc	108	78
2	18	2	아바타: 불가 재	판도	월드	/uplc	197	0
3	19	3	뫼비우스의 띠	너희	세상	/uplc	107	0
4	20	4	반지의 제왕: 두 개의 탑	9월의	2002	/uplc	179	0
5	21	5	극장판 주술회전: 시부야사변 X 사멸회	시부	2018	/uplc	87	0
6	22	6	극장판 체인소 맨: 레제런	압도	인기	/uplc	100	3
7	23	7	나우 유 씨 미 3	한때	나쁜	/uplc	112	0
8	24	8	정보원	공들	"나 5	/uplc	103	0
9	25	9	너의 999분	수원	을 거	/uplc	999	4
10	27	10	뽀로로 극장판 스위트캐슬 대모험	마녀	이번	/uplc	66	18
11	28	11	워킹드	천하	너로	/uplc	137	3
12	38	14	생존: 젓가락살인마	사람	절대	/uplc	120	26
13	40	15	우리...조금만 더 걸을까	요즘	남자	/uplc	300	38
14	43	16	test	test	test	/uplc	123	0

• 데이터 기반 내림차순 정렬 조회
'ORDER BY'를 활용하여
reserv_count 컬럼을 참조한 내림차
순 정렬 조회

책 내부에서 페이지별 4개씩 잘라
자리 지정

(index / 4)-1 로 지정

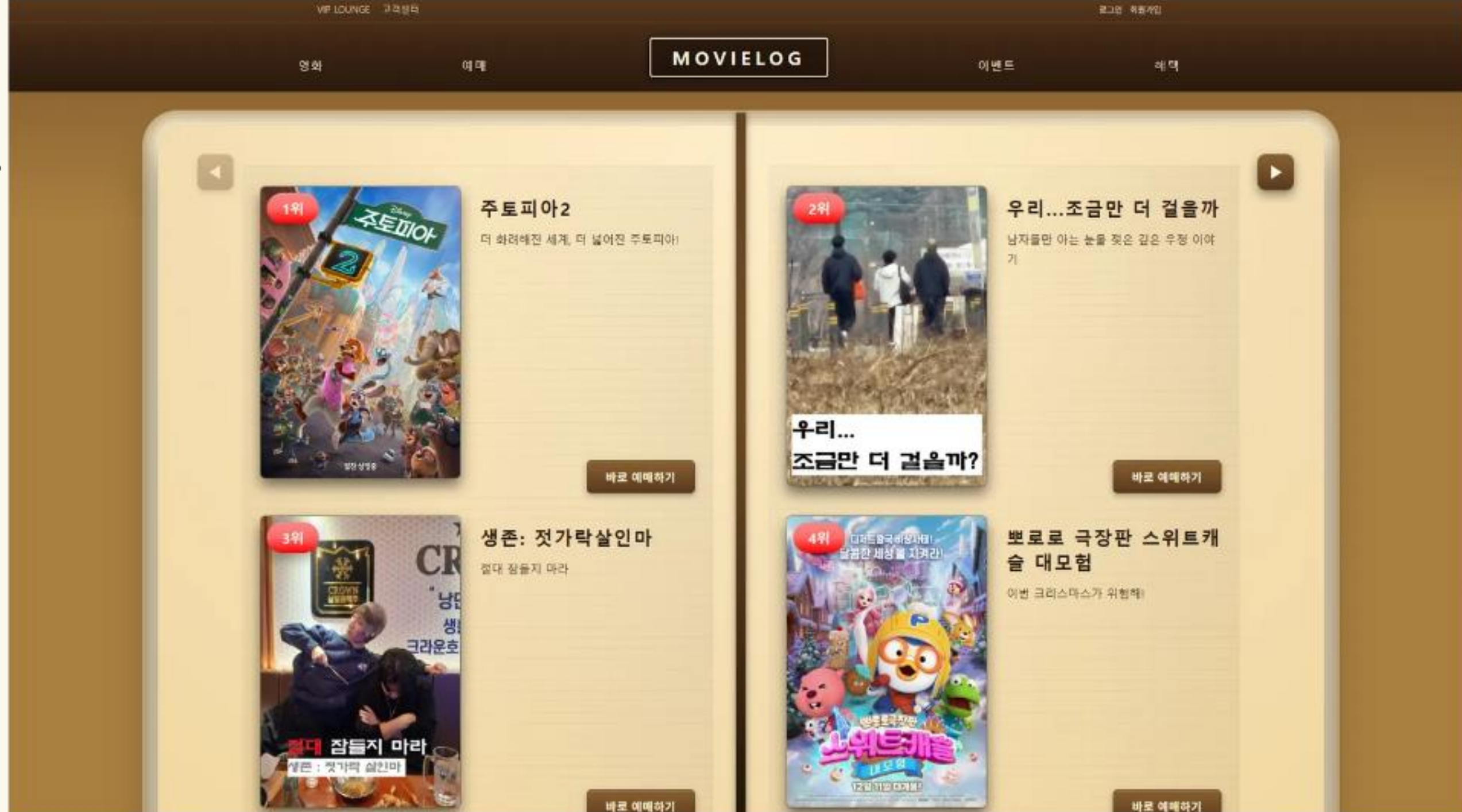
startIndex는 현재 page를 의미하여
page에 맞게 1위, 5위, 9위 ... 를 선정
하고 그에 맞는 영화 정보 송출

각 예매버튼 함수 실행시 해당 영화의
Primary Key를 넘긴 값을 바탕으로
'예매'화면의 해당 영화 스크롤로 자동
이동

• 페이지 넘김 버튼
삼항연산자를 통해 maxPage에 도달
하면 비활성화, 도달하지 않으면 활성화

```
<div className="book-wrapper">
  <div className="open-book">
    <div className="book-spine" />

    /* ===== 왼쪽 페이지 (1위 / 3위, 5위 / 7위 ...) ===== */
    <div className="page page-left">
      /* 상단: 현재 페이지의 첫 번째 순위 (1위, 5위, 9위 ...) */
      {leftTop && (
        <div className="movie-row">
          <div className="poster-wrap">
            <img
              src={`http://192.168.0.228:3000${leftTop.poster}`}
              alt={leftTop.title}
              className="poster-img"
            />
            /* 🟢 순위 숫자: 페이지에 따라 1,5,9... */
            <div className="rank-badge">
              {`${startIndex + 1}위`}
            </div>
          </div>
          <div className="desc">
            <h1>{leftTop.title}</h1>
            <span>{leftTop.short_description}</span>
          </div>
          <button
            className="quick-reserv"
            onClick={() => Reserve(leftTop.movie_id)}
          >
            바로 예매하기
          </button>
        </div>
      )}
    </div>
  </div>
  <div className="page page-right">
    /* 상단: 현재 페이지의 두 번째 순위 (2위, 6위, 10위 ...) */
    {rightTop && (
      <div className="movie-row">
        <div className="poster-wrap">
          <img
            src={`http://192.168.0.228:3000${rightTop.poster}`}
            alt={rightTop.title}
            className="poster-img"
          />
          /* 🟢 순위 숫자: 페이지에 따라 2,6,10... */
          <div className="rank-badge">
            {`${startIndex + 2}위`}
          </div>
        </div>
        <div className="desc">
          <h1>{rightTop.title}</h1>
          <span>{rightTop.short_description}</span>
        </div>
        <button
          className="quick-reserv"
          onClick={() => Reserve(rightTop.movie_id)}
        >
          바로 예매하기
        </button>
      </div>
    )}
  </div>
</div>
```



```
//페이지별 4개씩 잘라 쓰기
const pageSize = 4;
const startIndex = page * pageSize;
const pageMovies = movieData.slice(startIndex, startIndex + pageSize);

// 안전하게 자리 뽑아오기 (없을 수도 있으니까)
const leftTop = pageMovies[0]; // 1위, 5위, 9위 ...
const leftBottom = pageMovies[2]; // 3위, 7위 ...
const rightTop = pageMovies[1]; // 2위, 6위 ...
const rightBottom = pageMovies[3]; // 4위, 8위 ...

const maxPage = Math.max(0, Math.ceil(movieData.length / pageSize) - 1);
```


회원가입

```
//회원정보 조회
useEffect(() => {
  fetch("http://192.168.0.227:3000/userinfo")
    .then(response => response.json())
    .then(data => setUserData(data));
}, []);

//회원가입 시도
function handleJoin() {
  if (id !== "" && pw !== "" && name !== "") {
    if (!userData.some(item => item.id === id)) {
      if (pw === checkPw) {
        //회원가입 실행
        fetch("http://192.168.0.227:3000/join", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            userName: name,
            userId: id,
            userPw: pw
          })
        });
        alert("회원가입이 완료되었습니다. 로그인 화면으로 이동합니다.");
        navigate("/login");
      } else {
        alert("비밀번호 확인이 다릅니다.");
      }
    } else {
      alert("중복된 아이디입니다.");
    }
  } else {
    alert("입력하지 않은 정보가 있습니다.");
  }
}
```

- 사용자 id, pw, name 비교

1) id, pw, name 공백X

2) user테이블에 일치하는 id 없음

3) 비밀번호와 비밀번호 확인 값이 같음

->입력한 id, pw, name INSERT

로그인

```
//로그인 시도
function handleLogin() {
  const login = userData.find(item =>
    item.id === id && item.pw === pw);

  //로그인 성공
  if(login) {
    alert("로그인에 성공했습니다");
    navigate('/', { state:
      { name : login.name,
        id : login.id
      }
    });
  }else{
    alert("계정 정보가 없습니다");
  }
}
```



- 로그인 검증

user 테이블의 id, pw와
사용자가 입력한 id, pw를 비교
있으면 True, 없으면 False

<True일 경우>
메인화면으로 이동,
location state 활용하여
사용자 name, id 데이터를 같이 전달

예매 : 예매 정보 선택

```
//영화정보 조회
useEffect(() => {
  fetch("http://192.168.0.228:3000/movies")
    .then(response => response.json())
    .then(data => {
      const sorted = [...data].sort(
        (a, b) => a.screen_number - b.screen_number
      );
      setMovieData(sorted);
    });
}, []);

<label
  className="reservation-date-wrapper"
  onClick={(e) => {
    const input = e.currentTarget.querySelector(
      'input[type="date"]');
    if (input) input.focus(); // 모든 브라우저에서 지원
    if (input.showPicker) { // 크롬/엡지면 달력 바로 열기
      input.showPicker();
    }
  }}
>
<input
  id={item.movie_id}
  name="date"
  type="date"
  className="reservation-date-input"
  min={today}
  value={date[item.movie_id] || ''}
  onChange={(e) => setDate(prev => ({
    ...prev,
    [item.movie_id]: e.target.value
  })}
/>
</label>

{/* 시간별 버튼 */}
<div className="reservation-times">
  {item.start_time1 && (
    <span
      className="time-slot"
      onClick={() => moveSeat(
        item.title, item.start_time1,
        date[item.movie_id], item.screen_number,
        item.movie_id)}
    >
      {item.start_time1}
    </span>
  )}
</div>
</div>
```

- 상영관 기반 오름차순
예매누적수 반영이 안된 영화 정보를
호출하여 sort를 통해 상영관 오름차순
으로 정렬

- 날짜 선택 / 시간 선택시 즉시 이동
date를 입력한 후에 상영 시간을 선택
해야 좌석 선택 페이지로 이동

moveSeat 실행시 필요한 정보를 버
튼으로부터 모두 참조하여 다음 페이지
로 이동시 전송

- 정보 전송시 보안 고려
사용자가 URL에서 임의변경시 문제
가 되지 않는 예매정보는 useParams
를 통해 URL로 계속해서 보냄

임의변경시 문제를 야기하는
userInfo는 useLocation을 통해 계
속해서 보냄



```
//좌석선택으로 이동
function moveSeat(title, time, date, screen, movieId) {
  if (!userInfo.id) {
    alert("로그인 해주세요");
    return;
  }
  if (!date) {
    alert("날짜를 선택해주세요.");
    return;
  }
  navigate(`/seat/${title}/${time}/${date}/${screen}/${movieId}`, {
    state: {
      name: userInfo.name,
      id: userInfo.id
    }
  });
}
```


예매 : 좌석 선택

```
useEffect(() => {
  const seatNums = seatData.filter(item =>
    item.movie_name == reservInfo.title &&
    item.screen_num == reservInfo.screen &&
    item.date.split('T')[0] == reservInfo.date.split('T')[0] &&
    item.time == reservInfo.time
  )
  .flatMap(item =>
    item.seat_num.includes(',')
    ? item.seat_num.split(',')
    : [item.seat_num]
  )
  setReservedSeats(seatNums)
  console.log(seatNums)
}, [seatData])
```

```
const toggleSeat = (seat) => { //자리선택 행동

  if (reservedSeats.includes(seat)) {
    alert("이미 예매된 좌석입니다.");
    return;
  }
  if (selectedSeats.includes(seat)) {
    setSelectedSeats(selectedSeats.filter(s => s !== seat));
    console.log(selectedSeats.length)
  } else if (selectedSeats.length < personnel) {
    setSelectedSeats([...selectedSeats, seat]);
    console.log(selectedSeats.length)
    console.log(personnel)
  } else {
    alert("선택한 인원수보다 선택한 좌석이 많습니다")
  }
};
```

```
function personCount() {
  if (personnel >= seats.length - reservedSeats.length) {
    alert("인원수가 잔여석을 초과할 수 없습니다.")
    return;
  }
  setPersonnel(item => item += 1)
}
```

```
function personDecount() {
  if (personnel <= 0) {
    alert("인원수가 0보다 작을 수 없습니다.")
    return;
  }
  if (selectedSeats < personnel) {
    setPersonnel(item => item -= 1)
  } else {
    alert("먼저 좌석 선택을 취소해주세요.")
  }
}
```

- 중복 좌석 비활성화 구분
이미 예매된 좌석을 추려내서 reservedSeats에 set함

.flatMap을 통해 여러 좌석을 선택한 예매정보에서 ','을 기준으로 개별 좌석으로 분리하여 배열에 저장

reservedSeats의 값과 일치하는 좌석을 css 및 버튼 비활성화

예기치 못한 오류 발생 방지를 위해 '예매하기' 클릭시에도 예매된 좌석 추가 예매 방지

- 인원수 선택 기능
사용자가 인원수를 설정하여 그 수만큼 좌석 선택함.

목적.1 : 인원수 * 10p 지급을 위함
목적.2 : 총 결제금액 계산
목적.3 : 예기치 못한 오류 방지



```
function submit() { //예매 행동
  if (!selectedSeats[0]) {
    alert("좌석을 선택해주세요.")
    return;
  }

  const payment = confirm("결제 페이지로 이동하시겠습니까?")

  if (payment) {
    navigate(
      `/payment/${reservInfo.title}/${reservInfo.time}/${reservInfo.date}/`
    )
    {
      state: {
        name: userInfo.name,
        id: userInfo.id
      }
    }
  }
}
```


예매 : 결제

```
if (!selectedCard) {
    alert("결제 카드를 선택해주세요!");
    return;
}

const selectedCardInfo = cardData.find(
    (item) => String(item.card_defid) === selectedCard
);

fetch("http://192.168.0.228:3000/reservcount", {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
        movieId: reservInfo.movieId,
        addCount: seatCount
    })
})

fetch("http://192.168.0.228:3000/reserv", {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({
        date: reservInfo.date,
        movieName: reservInfo.title,
        movieTime: reservInfo.time,
        userName: userInfo.name,
        userId: userInfo.id,
        seat: reservInfo.seats,
        screen: reservInfo.screen,
        pickcount: seatCount,
        cardNumber: selectedCardInfo.card_num,
        cardBank: selectedCardInfo.card_bank,
        cardDate: selectedCardInfo.card_date
    })
})

.then(() => {
    return fetch(`http://192.168.0.228:3000/point/add/${userInfo.id}`, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ addPoint: seatCount * 10 })
    });
})

.then(res => res.json())
.then(() => {
    alert("결제에 성공했습니다. 마이페이지에서 내역확인이 가능합니다.");
    navigate("/", {
        state: { name: userInfo.name, id: userInfo.id }
    });
});
});
```

- 예매시 해당 영화 예매누적수 증가
reservInfo의 movie_id에 해당하는
영화의 좌석 수만큼 DB에 예매 누적수
증가 반영

- 예매 정보 POST 및 포인트 지급
결제 완료시 fetch, .then을 활용하여
모든 예매 전산처리 실행

- 등록 카드 부재시 페이지 이동
등록된 카드가 없는 채로 '결제하기' 실행
마이페이지의 카드등록 페이지로
이동 및 스크롤 이동

영화

예매

MOVIELOG

이벤트

레딕

결제 정보

영화 제목: 주토피아2

상영 날짜: 2025-12-17

상영 시간: 08:00

상영관: 1관

좌석: B3, B4

좌석 개수: 2개

총 결제 금액: 20000 원

결제 카드 선택

카드사 : 국민은행

카드 번호 : 1535-8647-6556-1512

만료일 : 12/27

결제하기

```
function submit() {
    if (cardData.length === 0) {
        alert("카드를 등록해주세요.");
        navigate("/myinfo", {
            state: { name: userInfo.name, id: userInfo.id }
        });
        return;
    }

    if (!selectedCard) {
        alert("결제 카드를 선택해주세요!");
        return;
    }
}
```


영화 : 검색 기능

```
48   return (
49     <>
50     <MainHeader />
51
52     {/* 검색창 영역 */}
53     <div className="movie-search-box">
54       <input
55         type="text"
56         placeholder="영화 제목 검색"
57         value={search}
58         onChange={(e) => setSearch(e.target.value)}
59       />
60       <button onClick={handleSearch}>검색</button>
61     </div>
```

```
27   // 검색해서 스크롤 이동
28   const handleSearch = () => {
29     if (!search.trim()) return;
30
31     // 검색어와 영화 제목 대조
32     const foundMovie = movies.find((m) =>
33       m.title.toLowerCase().includes(search.toLowerCase())
34     );
35
36     if (foundMovie) {
37       const movieId = foundMovie.movie_id;
38       const targetRef = movieRefs.current[movieId];
39
40       if (targetRef) {
41         targetRef.scrollIntoView({ behavior: "smooth", block: "start" });
42       }
43     } else {
44       alert("검색한 영화가 목록에 없습니다.");
45     }
46   };
```

• 영화 목록 표시

DB에 저장되어있는 영화의 정보를 표시, 예매 누적수 실시간 확인 가능

• 검색 기능

input에서 받은 text와 영화 제목을 비교해서 일치하는 값이 있으면 foundMovie가 True

movie_id를 비교해서 targetRef도 True라면,

scrollIntoView를 통해 movie_id가 일치하는 곳으로 스크롤 이동



이벤트 및 혜택

```
import MainHeader from '../Main/MainHeader.jsx'
import { useState, useEffect } from 'react'

import '../css/Event.css'

function Event(){
  const [eventData, setEventData] = useState([])

  //이벤트 정보 조회
  useEffect(()=>{
    fetch("http://192.168.0.228:3000/eventinfo")
      .then(response=>response.json())
      .then(data=>setEventData(data))
  },[])

  return(
    <>
      <MainHeader />
      {/* 이벤트 포스터 */}
      {eventData.map((item)=>{
        <div key={item.defid}>
          <img src={`http://192.168.0.228:3000/${item.poster_path}`}
            className="event-poster"
            alt={item.poster_name} />
        </div>
      })}
    </>
  )
}

export default Event;
```

```
import MainHeader from '../Main/MainHeader.jsx'
import { useState, useEffect } from 'react'

import '../css/Benefit.css'

function Benefit(){
  const [benefitData, setBenefitData] = useState([])

  //혜택 정보 조회
  useEffect(()=>{
    fetch("http://192.168.0.228:3000/benefitinfo")
      .then(response =>response.json())
      .then(data => setBenefitData(data))
  },[])

  return(
    <>
      <MainHeader />
      {/* 혜택 포스터 */}
      {benefitData.map((item)=>{
        <div key={item.defid}>
          <img src={`http://192.168.0.228:3000/${item.poster_path}`}
            className="benefit-poster"
            alt={item.poster_name} />
        </div>
      })}
    </>
  )
}

export default Benefit;
```

- 포스터 업로드 형식

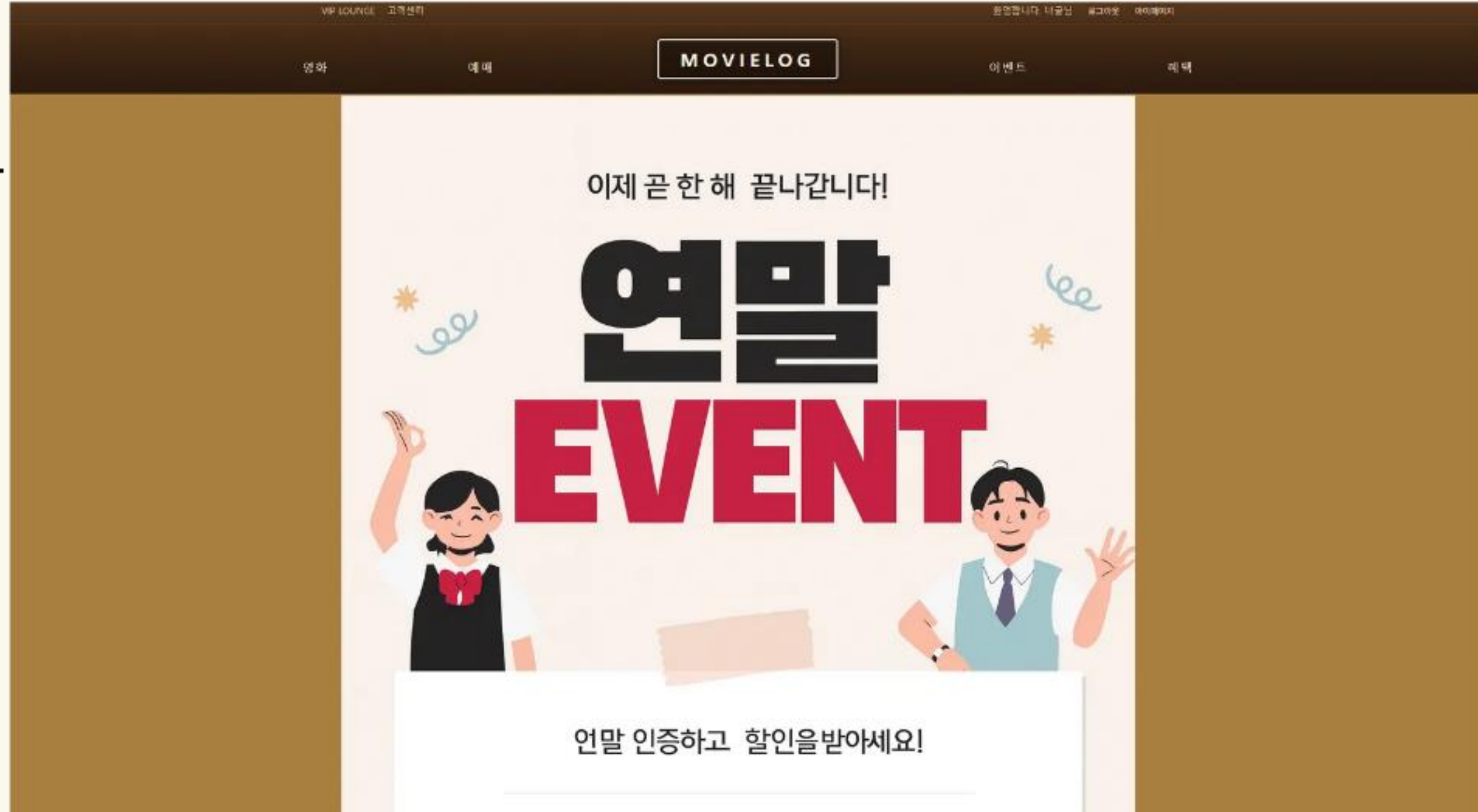
관리자가 별도 입력 없이 포스터를 제작하거나 외주를 받아서 2:3 규격의 이미지를 업로드하도록 설계.

- 이벤트, 혜택 동일한 원리

DB에서 해당 사진의 경로를 호출해 송출하며 관리자 페이지에서 등록시 임의로 지정한 경로로 사진을 저장하고 경로를 추가, 변경하는 구조.

- 이점

사용자의 혜택정보 열람에 무리가 없는 선에서 관리자의 관리 및 유지보수의 효율성이 극대화 됨



마이페이지-내 정보

```
//헤달 유저 프로필 사진 조회
useEffect(() => {
  setLoggedInName(userInfo.name);

  fetch(`http://192.168.0.227:3000/userprofile/${userInfo.id}`)
    .then(res => res.json())
    .then(data => {
      const profilePath = Array.isArray(data) && data.length > 0
        ? data[0].profile : null;
      if (profilePath) {
        setProfileImg(`http://192.168.0.227:3000${profilePath}`);
      }
    });
}, []);

//예매 내역 조회
useEffect(() => {
  fetch(`http://192.168.0.227:3000/seatlist/${userInfo.id}`)
    .then(res => res.json())
    .then(data => setSeatData(data))
}, []);

//포인트
useEffect(() => {
  fetch(`http://192.168.0.227:3000/point/${userInfo.id}`, {
  })
    .then(res => res.json())
    .then(data => setPoint(data));
}, []);

//프로필 사진 변경
async function handleProfileChange(e){
  const file = e.target.files[0];
  if (!file) return;

  const formData = new FormData();
  formData.append("profile", file);
  formData.append("userId", userInfo.id);

  const res = await fetch("http://192.168.0.227:3000/updateProfile", {
    method: "PUT",
    body: formData
  });

  const data = await res.json();

  if (data.success) {
    setProfileImg(`http://192.168.0.227:3000${data.profile}`);
  } else {
    alert("업로드 실패");
  }
}
```

- 프로필 사진 조회

state에 있는 로그인 된 사용자 id로 db 테이블의 profile 주소를 받아서 img주소로 읽음

- 프로필 사진 변경

multer함수를 활용하여 지정한 파일 저장 경로로 이미지를 업로드

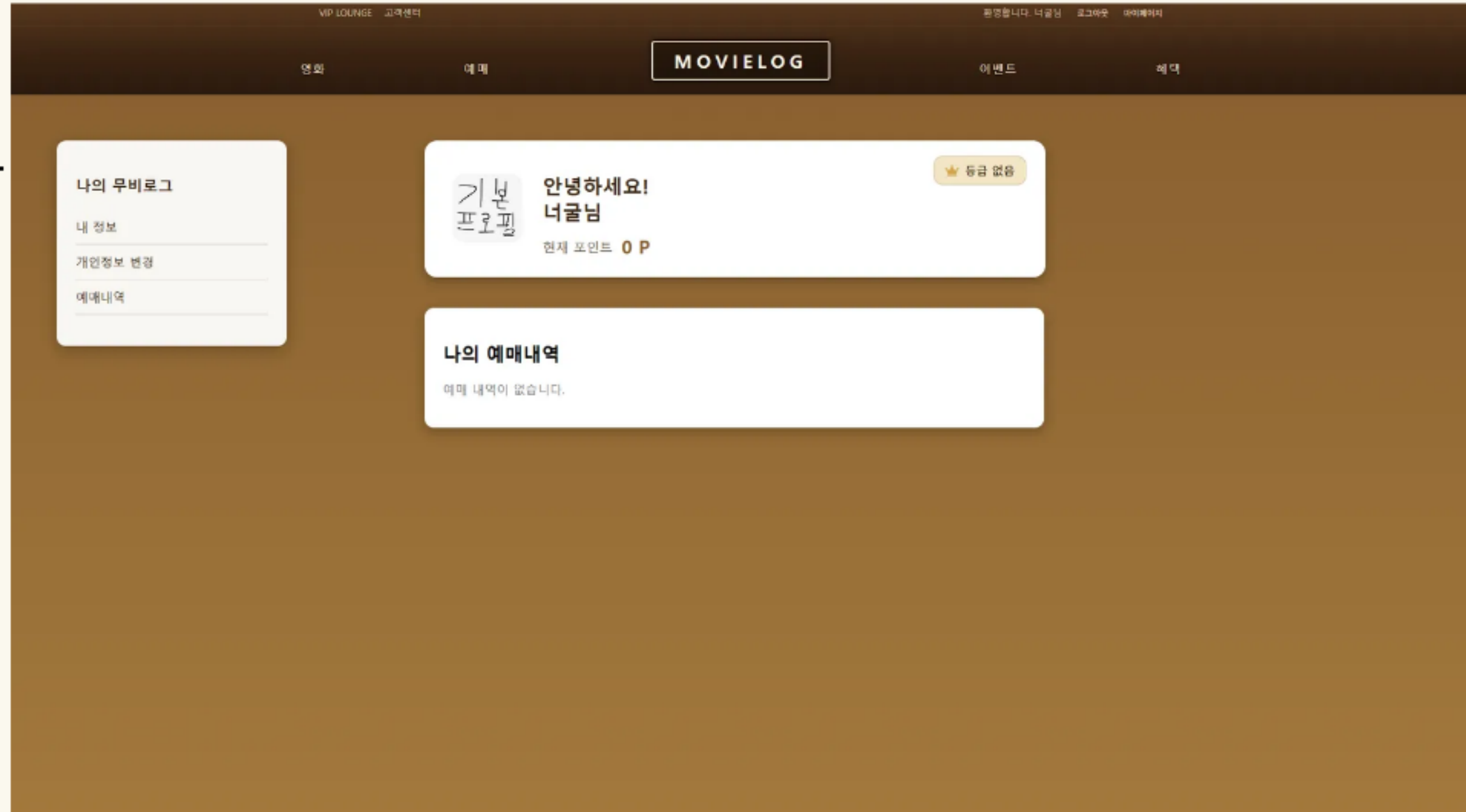
db 테이블에 업로드한 profile이미지 주소 업데이트

- 포인트 조회

예매할 때 누적된 포인트를 표시하고 포인트 값에 따라 등급 변화

- 나의 예매내역 조회

사용자 id를 통해 예매 내역을 내림차순으로 정렬하여 표시



- 변경된 내 정보



마이페이지-이름/비밀번호 변경

```
//이름 변경
function handleChangeName() {
  if (newName !== confirmName) {
    alert("이름이 일치하지 않습니다.");
    return;
  }
  fetch("http://192.168.0.227:3000/changeName", {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      userId: userInfo.id,
      newName: newName
    })
  })
  alert("이름이 변경되었습니다. 다시 로그인해주세요.")
  navigate('/', {
    state: {
      name: null,
      id: null
    }
  })
};
```

```
//비밀번호 변경
function handleChangePassword() {
  if (newPw !== confirmPw) {
    alert("비밀번호 확인이 일치하지 않습니다.");
    return;
  }
  fetch("http://192.168.0.227:3000/changePassword", {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      userId: userInfo.id,
      newPassword: newPw
    })
  })
  alert("비밀번호가 변경되었습니다. 다시 로그인해주세요.")
  navigate('/', {
    state: {
      name: null,
      id: null
    }
  })
};
```

• 이름 변경

새 이름과 이름 확인을 비교

일치 하면 현재 state에 있는 id와
새로 입력한 newName을 넘기고 id에
일치하는 user테이블 컬럼 업데이트

• 비밀번호 변경

새 비밀번호와 비밀번호 확인을 비교
일치 하면 현재 state에 있는 id와
새로 입력한 newPw를 넘기고 id에
일치하는 user테이블 컬럼 업데이트

개인정보 변경

이름 변경

새 이름

이름 확인

변경하기

비밀번호 변경

새 비밀번호

비밀번호 확인

변경하기

마이페이지-카드등록

```
//카드 등록
function cardSubmit() {
  const user = userData.find(item=>item.id == userInfo.id)

  const cardRegex = /^\\d{4}-\\d{4}-\\d{4}-\\d{4}$/;
  const cardDateRegex = /^([0-9]{1}[0-2])[\\d{2}$/;

  if(cardRegex.test(newCard)){
    if(cardDateRegex.test(newCardDate)){
      if(!bank == ""){
        fetch("http://192.168.0.227:3000/newcard",{
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            card: newCard,
            cardDate: newCardDate,
            userId: userInfo.id,
            defid: user.defid,
            bank: bank,
            name: newCardName
          })
        })
      }
      alert("카드 등록이 완료되었습니다.")
      setNewCardName("")
      setNewCard("")
      setNewCardDate("")
      navigate('/myinfo', {
        state: {
          name: userInfo.name,
          id: userInfo.id
        }
      })
    }
  }else{
    alert("은행을 선택해주세요.")
  }
}else {
  alert("유효기간에 '/' 포함하여 올바르게 입력해주세요.")
}
}else{
  alert("카드 번호에 '-' 포함하여 올바르게 입력해주세요.")
}
}

//카드 별명 수정
function cardNameEdit(defid, name) {
  const newName = prompt("새로운 카드 별명을 입력해주세요.", name)
  if(newName){
    fetch("http://192.168.0.227:3000/cardnameupdate",{
      method: "PUT",
      headers: {"content-type": "application/json"},
      body: JSON.stringify({
        cardId: defid,
        cardName: newName
      })
    })
  }
}
```

• 카드 등록

1) 카드 번호와 유효기간을 형식에 맞게 입력
카드사는 option태그를 사용하여 리스트에서 선택

2)입력한 카드 정보와 userid와 defid(카드 고유 id)
를 db테이블에 저장

• 카드 별명 수정

defid와 새 이름을 넘기고, defid에 맞는
컬럼의 카드 이름을 수정

• 카드 삭제

defid를 넘겨서 해당하는 defid 카드의
컬럼을 테이블에서 삭제

카드 등록

카드 별명

카드 별명

카드 번호

0000-0000-0000-0000

유효기간

MM/YY

카드사

은행 선택

카드 등록

내 카드

나라사랑카드

카드 별명 수정

카드사 : 국민은행

카드 번호 : 1535-8647-6556-1512

만료일 : 12/27

삭제

//카드 삭제

```
function cardDelete(defid) {
  const yes = confirm("진짜 지울거니?")
  if(yes){
    fetch("http://192.168.0.227:3000/carddelete",{
      method: "DELETE",
      headers: {"content-type": 'application/json'},
      body: JSON.stringify({defid : defid})
    })
  }
}
```


마이페이지-예매내역

```
useEffect(() => {
  fetch(`http://192.168.0.227:3000/seatlist/${userInfo.id}`)
    .then(response => response.json())
    .then(data => setSeatData(data))
}, [seatData]);
```

```
<div className="reserved-box">
  <h2>나의 예매내역</h2>

  {seatData.length === 0 ? (
    <p className="empty">예매 내역이 없습니다.</p>
  ) : (
    <div className="reserve-list">
      {seatData.map((item) => (
        <div key={item.seat_id} className="reserve-card">

          <div className="reserve-header">
            <h3 className="reserve-title">{item.movie_name}</h3>
            <span className="reserve-num">예매번호 : 0{item.screen_num}-{item.seat_id * item.seat_id}</span>
          </div>

          <div className="reserve-body">
            <div className="reserve-row">
              <span className="label">상영관</span>
              <span className="value">{item.screen_num}관</span>
            </div>

            <div className="reserve-row seat-row">
              <span className="label">좌석</span>
              <span className="value seat-value">{item.seat_num.split(",").join(", ")}</span>
            </div>

            <div className="reserve-row">
              <span className="label">관람일시</span>
              <span className="value">{item.date.slice(0, 10)} {item.time}</span>
            </div>
          </div>
        </div>
      )
    )}
    </div>
  )}
</div>
```



- 예매내역 확인

예매 테이블에서 user_id와 일치하는 정보를 전부 표시

내림차순 정렬로 최근에 예매한 내역이 화면 위에 오도록 표시

좌석 개수가 많을 때 비율이 깨지지 않도록
split(",").join(",")을 통해 줄바꿈 형식 추가

VIP : LOUNGE

```
const WebSocket = require("ws");

const wss = new WebSocket.Server({ port: 3001 });

// 메모리에 저장할 임시 채팅 기록
let chatHistory = [];

console.log("🔥 WebSocket 서버 실행 중... ws://localhost:3001");

wss.on("connection", (ws) => {
  console.log("✨ 클라이언트 연결됨");

  // 새로 입장한 클라이언트에게 과거 메시지 전송
  chatHistory.forEach(msg => {
    ws.send(JSON.stringify(msg));
  });

  ws.on("message", (msg) => {
    console.log("■ 받은 메시지:", msg.toString());

    let parsed;
    try {
      parsed = JSON.parse(msg);
    } catch (e) {
      console.log("JSON 파싱 실패:", msg.toString());
      return;
    }

    // 메모리에 저장
    chatHistory.push(parsed);

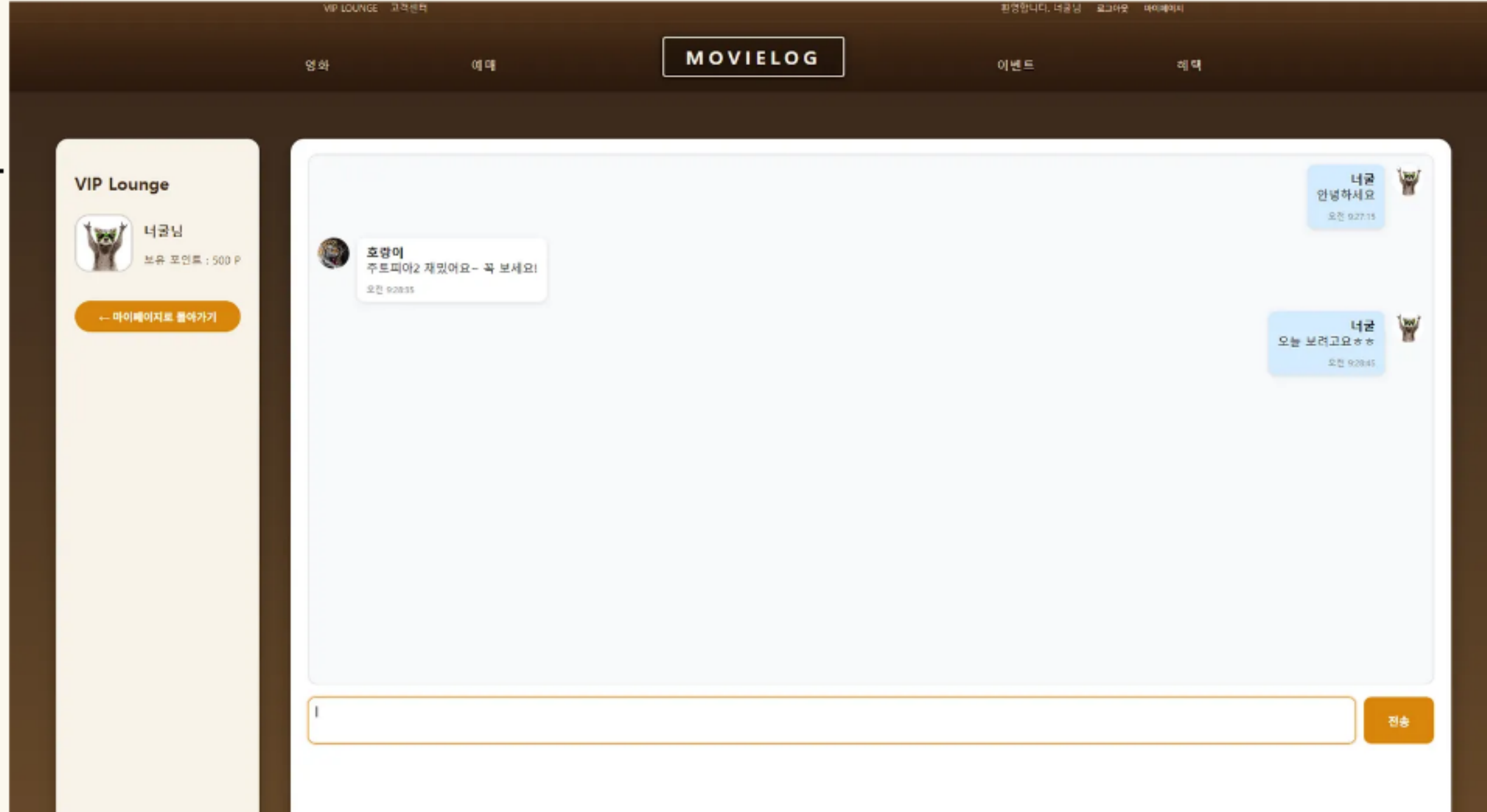
    // 전체 클라이언트에게 브로드캐스트
    wss.clients.forEach(client => {
      if (client.readyState === WebSocket.OPEN) {
        client.send(JSON.stringify(parsed));
      }
    });
  });

  ws.on("close", () => {
    console.log("✖ 클라이언트 연결 종료");
  });
});
```

- Web Socket 서버 활용
접속한 모든 사용자와 연결을 유지하
며, 한 사용자가 보낸 메시지를 즉시 모
든 사용자에게 실시간으로 전달하는
wsServer를 활용하여 실시간 양방향
통신이 가능한 채팅방 개설

- 입장 권한
로그인 필수, 500 point 이상 보유자
(예매 50석 이상) 사용자만 이용 가능.
영화 매니아들이 평론을 나눌 수 있는
공간으로 활용

- In-Memory 활용 임시 저장
채팅 기록은 영구정보보다 즉시성을 우선
하므로 DB를 활용하지 않고 서버의 임
시 메모리에 저장하며 서버가 닫히면
자동 삭제되므로 별도 관리 작업이 필
요하지 않아 관리 및 데이터 저장공간
효율이 극대화 됨.



```
useEffect(() => {
  fetch(`http://192.168.0.228:3000/userprofile/${userId}`)
    .then(res => res.json())
    .then(data => {
      const profile = Array.isArray(data) && data.length > 0 ? data[0] : null;
      if (profile) {
        setProfileImg(`http://192.168.0.228:3000/${profile}`);
      }
    });
}, [userId]);

useEffect(() => {
  if (!userName || !userId) {
    alert("로그인이 필요합니다.");
    navigate("/");
    return;
  }

  if (userPoint < 500) {
    alert("VIP 라운지는 포인트 500점 이상부터 이용 가능합니다.");
    navigate("/mypage", { state });
  }
}, [userName, userId, userPoint]);
```

```
useEffect(() => {
  const host = window.location.hostname;
  const wsUrl = host.startsWith("192.168")
    ? "ws://192.168.0.228:3001"
    : "ws://112.218.47.101:3001";

  const ws = new WebSocket(wsUrl);
  wsRef.current = ws;

  ws.onmessage = (event) => {
    try {
      const json = JSON.parse(event.data);
      setMessages(prev => [...prev, json]);
    } catch {
      console.log("일반 메세지:", event.data);
    }
  };

  return () => ws.close();
}, []);
```


관리자 : 진입 및 계정관리

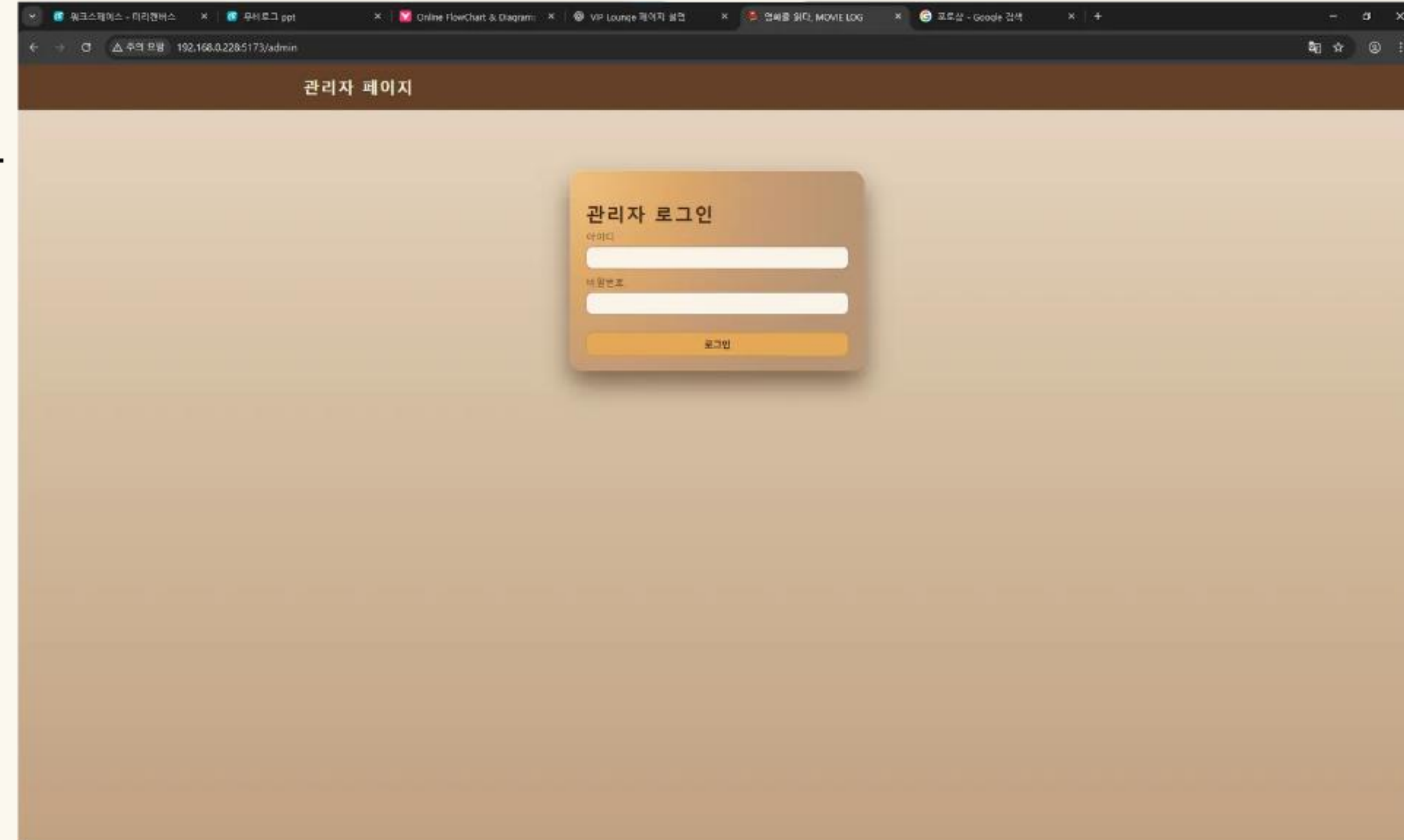
192.168.0.228:5173/admin

- 1차 보안 : 주소창에 입력시 접근 별도의 진입 버튼 및 힌트를 Frontend에 지원하지 않아 관리자페이지의 존재를 인지하는 대상만 접근 가능

```
return (  
  <div className="admin-page admin-login-page">  
    <AdminHead />  
  
    <div className="admin-main">  
      <div className="admin-login-card">  
        <h2 className="admin-title">관리자 로그인</h2>  
  
        <div className="admin-form">  
          <div className="admin-form-row">  
            <label className="admin-label">아이디</label>  
            <input  
              className="admin-input"  
              type="text"  
              value={id}  
              onChange={(e) => setId(e.target.value)}  
            />  
          </div>  
  
          <div className="admin-form-row">  
            <label className="admin-label">비밀번호</label>  
            <input  
              className="admin-input"  
              type="password"  
              value={pw}  
              onChange={(e) => setPw(e.target.value)}  
            />  
          </div>  
  
          <button  
            className="admin-button admin-button--primary admin-login-button"  
            onClick={login}>  
            로그인  
          </button>  
        </div>  
      </div>  
    </div>  
  </div>  
)
```

- 2차 보안 : 관리자 계정 관리 메뉴얼 웹에서 회원가입 및 계정 찾기 기능을 지원하지 않고 오직 데이터베이스를 통해서만 관리자 계정 관리 가능. 계정 관리 필요시 데이터베이스 담당자와 유선 및 무선으로 내통하여 신분확인 후 관리 가능.

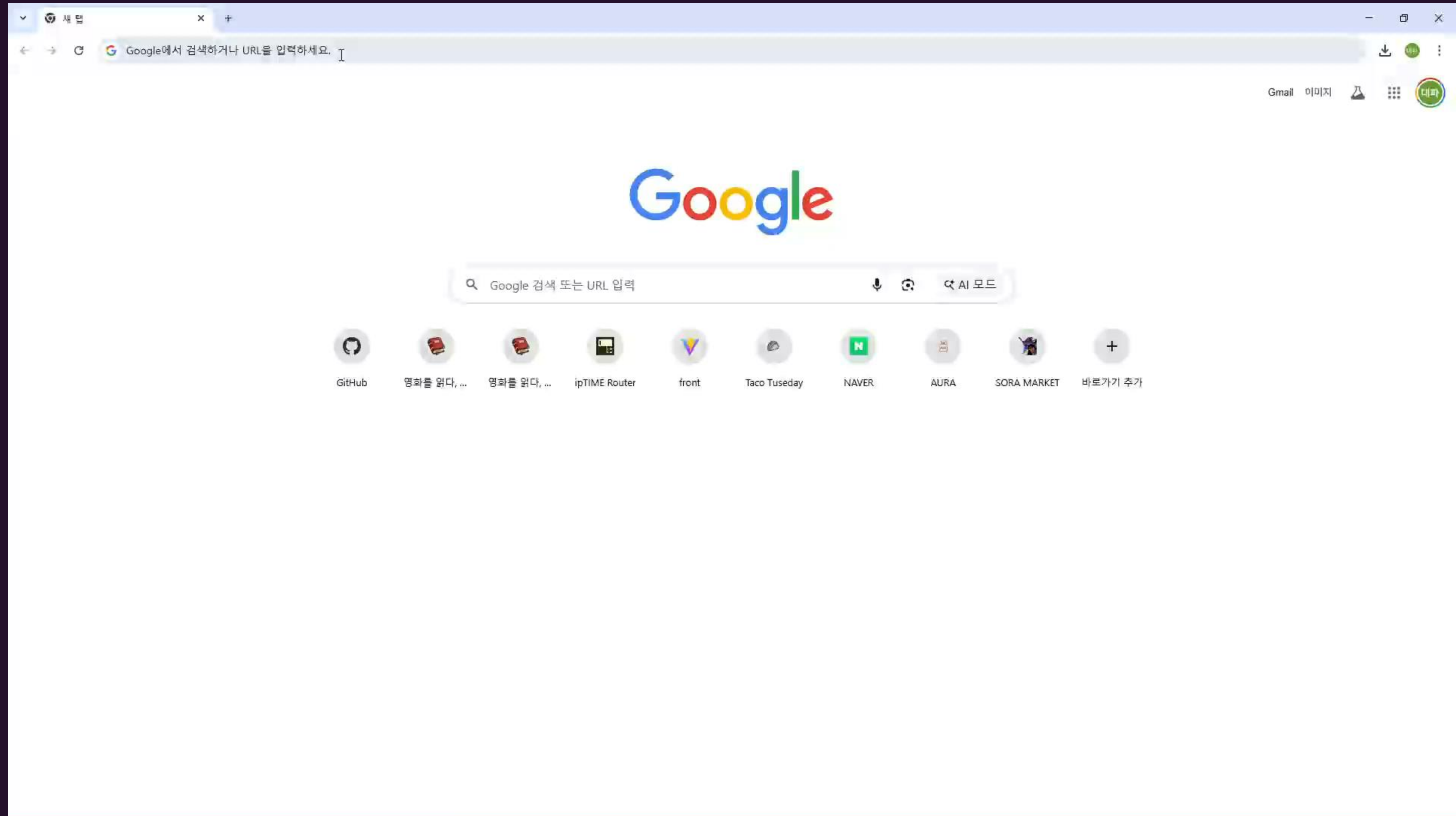
- 페이지 이탈시 자동 로그아웃 로그인 상태가 유지되면 외부인의 관리자 접근 가능성이 높으므로 별도로 로그인 유지 기능을 만들지 않고 일시적 접근의 형태를 유지함



```
const [id, setId] = useState("")  
const [pw, setPw] = useState("")  
const [adminData, setAdminData] = useState([])  
  
useEffect(() => {  
  fetch("http://192.168.0.228:3000/admin")  
    .then(response => response.json())  
    .then(data => setAdminData(data))  
}, [])  
  
function login() {  
  const login = adminData.find(item =>  
    item.admin_id == id && item.admin_pw == pw  
  )  
  if (login) {  
    alert("로그인 되었습니다.")  
    navigate('/admin/management', { state: id })  
  } else {  
    alert(  
      "경고 : 관리자 정보가 존재하지 않습니다. 관리자 외에 접근할 수 없습니다."  
    )  
  }  
}
```

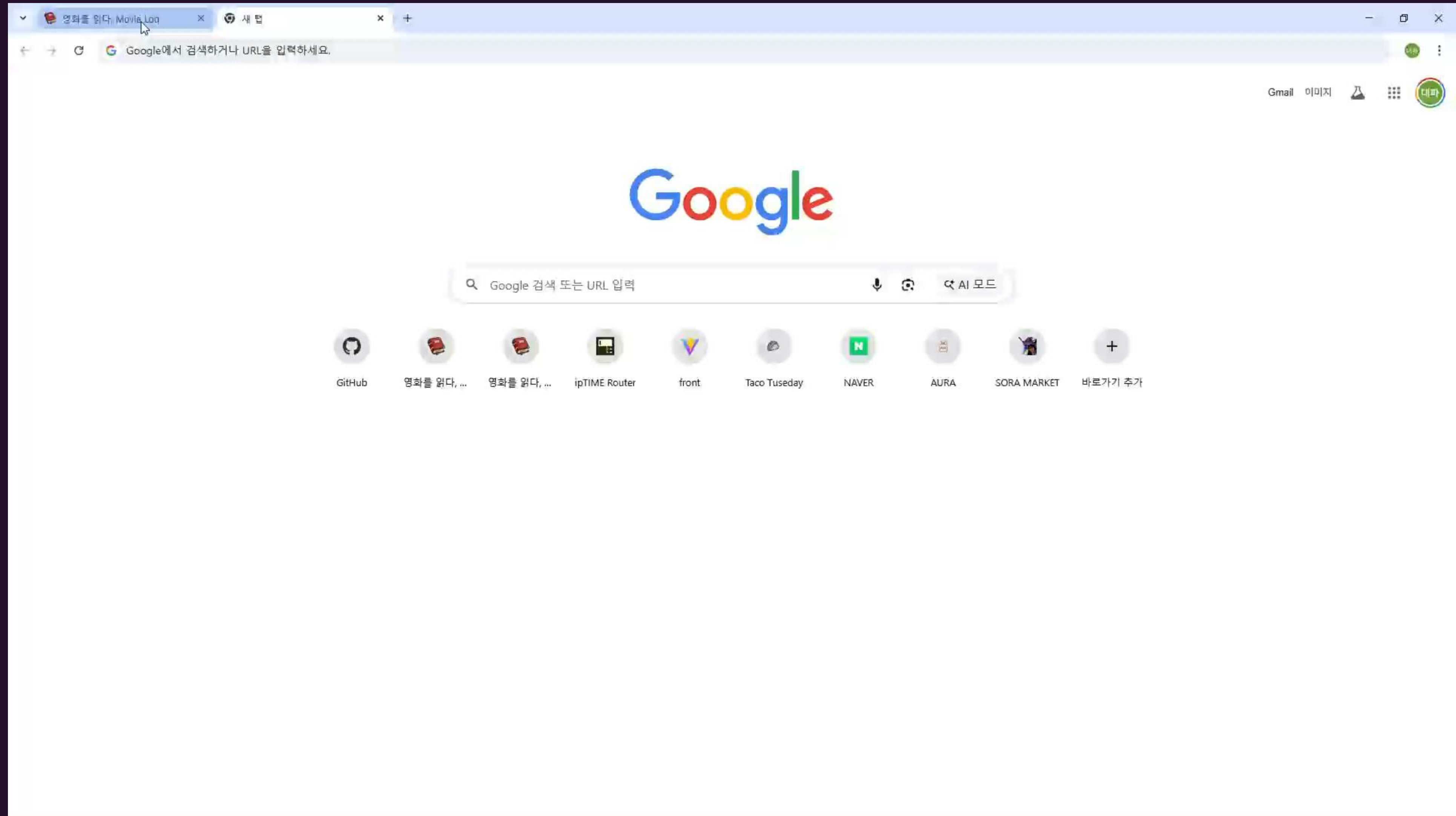

영화를 읽다, MOVIE LOG

사용자 시연 영상

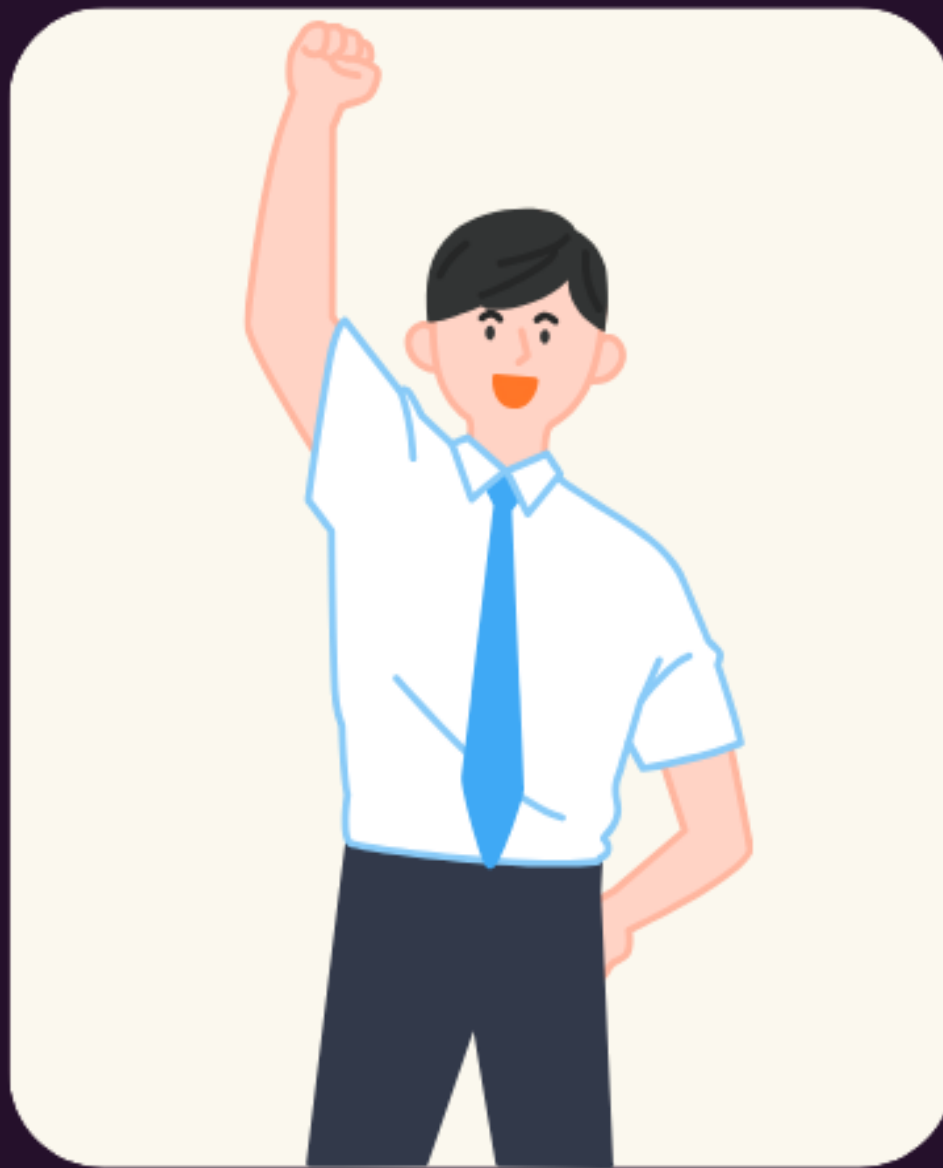


영화를 읽다, MOVIE LOG

관리자 시연 영상



팀 자체평가



팀장 : 김찬

이번 프로젝트를 통해 프론트엔드, 백엔드, 데이터베이스가 서로 어떻게 연결되어 동작하는지 이해할 수 있었습니다. 직접 사용자, 예매, 영화 등의 정보를 관리하게 되면서 DB 테이블은 어떻게 구성해야되고, 어떻게 관리해야하는지도 알 수 있었고, 특히 데이터 전달 과정에서 발생한 오류를 해결하며 디버깅의 중요성과 문제 해결 능력을 기를 수 있었습니다. 초기에 데이터베이스 구조와 API 명세를 충분히 정리하지 못해 개발 과정에서 수정이 반복된 점이 아쉬웠지만, 그 과정 속에서 팀원들과 더욱 활발하게 소통을 하며 상황에 맞게 유연적으로 개발하게 되면서, 상황에 맞춰서 개발하는 능력을 더욱 키울 수 있었습니다. 책과 영화라는 아이디어를 활용하면서 참신한 아이디어로 프로젝트를 만들게 된 것이 이번 프로젝트에게 가장 큰 성과인 것 같습니다. 향후 예매 인원 제한, 좌석 관리, 결제 기능 등 실제 서비스에 필요한 요소들을 보완하여 지금보다 완성도를 더욱 높일 수 있다고 생각합니다.

팀 자체평가



팀원 : 유조현

기존의 영화 예매 페이지는 이미 많지만 남녀노소 활용하기엔 직관적이지 않은 구조와 디자인, 복잡한 절차가 많습니다. 때문에 저희 '무비로그'는 남녀노소 활용할 수 있도록 직관적이고 단순한 구조로 설계하였습니다. 구조와 디자인을 단순화하여 접근성을 높였으며 '책속의 영화'라는 의미 깊고 감성적인 기획으로서 타 영화 예매처 대비 경쟁력을 갖추고 VIP 전용 채팅 시스템을 통해 사용자들에게 프라이빗한 공간의 커뮤니케이션을 제공하여 영화를 좋아하는 사람들에게 특수한 유대감을 형성하고 제한된 공간에 대한 호기심을 유발하여 VIP가 되기 위해 예매를 유도하는 마케팅적 효과도 지니고 있습니다. 추가적으로 '무비로그'에서만 상영하는 특별기획 영화가 있어 특수한 경쟁력을 다수 보유하여 새로운 예매처로서 시장에 자리매김할 수 있을 것으로 보입니다. 관리자 페이지에서도 보안상 문제가 있는 관리자 계정 관리 외에는 데이터베이스에 직접 관여하지 않아도 직관적이고 자유롭게 관리할 수 있는 시스템 또한 관리에 용이합니다. 불필요한 절차와 디자인을 최소화하고 필수적인 기능을 바탕으로 사람들이 재미를 느낄 수 있는 요소들이 많아 타겟층을 넓게 확보할 수 있겠습니다.

팀 자체평가



팀원 : 곽재범

영화예매 사이트라는 흥미로운 주제를 모티브로 만들었습니다. 전체적인 웹구조가 단순하여 필요요소가 보기 쉽고 작동이 용이하게 배열됐습니다. 버튼이나 메뉴를 열람하는 창이 조화롭게 배치 되었고 버튼 기능도 정확히 작동하고 있어 설계시 의도한 정확한 기능을 구현하고 있습니다. 전체적으로 조화가 잘 이뤄졌으며 사용자가 복잡한 과정없이 필요한 요소를 즉시 알아볼 수 있도록 웹이 구성되었습니다. 또한 사용자 입장에서 이벤트나 고객센터에 직접 접근할 수 있게 메인 페이지에 해당 요소를 가미한건 웹 설계자가 사용자 관점에서 웹을 설계하려는 노력을 엿볼 수 있습니다. 팀원들이 협동하여 의도한 대로 웹구조가 정확히 이뤄졌으며 웹페이지도 목적에 맞게 화면이 구성되었습니다. 특히 고객의 관점에서 고객중심의 여러 메뉴(이벤트, 혜택, 고객센터)등을 메인페이지에 마련하여 사용자 중심의 편의성 및 접근성을 높였다는 점이 눈에 띄게 두드러집니다. 목적에 맞는 정확한 웹페이지가 만들어졌으며 사용자 입장, 관리자 입장에서 필수 요소를 쉽게 접근 및 관리가 가능하게 웹이 설계되었다고 판단됩니다. 데이터베이스나 백엔드도 정보관리를 지원하고 있어 프론트엔드 부터 백엔드까지 그 구조와 기능이 합목적성에 비추어 정확하게 표현하고자 하는 바를 수행 했다고 판단됩니다.

2팀 여러분 모두 수고하셨습니다.

이상으로 마치겠습니다 && 감사합니다



Call

010-8868-1937



E-mail

yjh0207m@naver.com



Website

<http://192.168.0.228:5173/>

